

MuPDF — Tutoriel complet C++

Architecture, types, fonctions et patterns pour créer, lire et modifier des PDF.

1. Architecture

MuPDF est organisé en deux couches :

- **Couche Fitz (fz_*)** — API générique haut niveau. Rendu, extraction de texte, images. Supporte PDF, XPS, EPUB, HTML. C'est la couche utilisée 90% du temps.
- **Couche PDF (pdf_*)** — API bas niveau spécifique PDF. Manipulation des objets internes (dictionnaires, streams). Nécessaire pour créer ou modifier la structure d'un PDF.

Hierarchie des objets

```
fz_context      ← racine de tout, un par thread
├── fz_document  ← fichier ouvert (PDF, XPS...)
│   └── fz_page  ← une page du document
│       ├── fz_stext_page ← texte structuré
│       ├── fz_pixmap    ← image rasterisée
│       └── fz_display_list ← liste d'affichage (cache)
```

Cycle de vie obligatoire

Chaque objet créé doit être libéré avec sa fonction `fz_drop_*` correspondante.

```
fz_context *ctx = fz_new_context(NULL, NULL, FZ_STORE_UNLIMITED);
fz_register_document_handlers(ctx); // enregistre les formats supportés

fz_document *doc = fz_open_document(ctx, "fichier.pdf");

// ... travail ...

fz_drop_document(ctx, doc);           // libérer dans l'ordre inverse
fz_drop_context(ctx);                 // ctx toujours en dernier
```

Gestion d'erreurs — fz_try/catch

MuPDF n'utilise pas les exceptions C++. Il a son propre système basé sur `setjmp/longjmp`.

```
fz_try(ctx) {
    doc = fz_open_document(ctx, "fichier.pdf");
    page = fz_load_page(ctx, doc, 0);
}
```

```

fz_always(ctx) {
    // exécuté TOUJOURS (succès ou erreur) – idéal pour libérer
    fz_drop_page(ctx, page);
}
fz_catch(ctx) {
    fprintf(stderr, "Erreur : %s\n", fz_caught_message(ctx));
    return -1;
}

```

⚠ Ne jamais appeler `return` ou `throw` à l'intérieur d'un bloc `fz_try`.

2. Types fondamentaux

Contexte et document

Type	Rôle	Création
<code>fz_context</code>	Allocateur, gestion erreurs, cache. 1 par thread.	<code>fz_new_context()</code>
<code>fz_document</code>	Document ouvert (tout format)	<code>fz_open_document()</code>
<code>pdf_document</code>	Accès bas niveau spécifique PDF	<code>pdf_open_document()</code> ou cast
<code>fz_page</code>	Page générique	<code>fz_load_page()</code>
<code>pdf_page</code>	Page PDF avec accès objets internes	<code>pdf_load_page()</code>

Géométrie

Type	Contenu	Usage
<code>fz_rect</code>	float x0, y0, x1, y1	Boîte englobante
<code>fz_irect</code>	int x0, y0, x1, y1	Rect en pixels entiers
<code>fz_matrix</code>	6 floats (a,b,c,d,e,f)	Transformation 2D
<code>fz_point</code>	float x, y	Coordonnée 2D
<code>fz_quad</code>	4 <code>fz_point</code>	Quadrilatère (résultats de recherche)

```

fz_matrix identity = fz_identity;
fz_matrix scale    = fz_scale(2.0f, 2.0f);
fz_matrix rot      = fz_rotate(90);
fz_matrix trans    = fz_translate(10, 20);
fz_matrix combined = fz_concat(scale, rot);

// Rasteriser à 150 DPI (PDF natif = 72 DPI)
fz_matrix m = fz_scale(150.0f/72.0f, 150.0f/72.0f);

```

Contenu

Type	Rôle
<code>fz_pixmap</code>	Image rasterisée en mémoire (RGBA, RGB, Gray...)
<code>fz_buffer</code>	Buffer de bytes
<code>fz_stext_page</code>	Texte structuré extrait d'une page
<code>fz_stext_block</code>	Bloc de texte (paragraphe ou image)
<code>fz_stext_line</code>	Ligne de texte dans un bloc
<code>fz_stext_char</code>	Caractère individuel avec position et fonte
<code>fz_display_list</code>	Liste d'affichage mise en cache
<code>fz_device</code>	Interface de rendu

Types PDF bas niveau

Type	Rôle
<code>pdf_obj</code>	Objet PDF générique (int, float, string, array, dict, stream)
<code>pdf_graft_map</code>	Carte pour copier objets entre documents
<code>pdf_write_options</code>	Options de sauvegarde

3. Lire un PDF

Ouverture et informations

```
fz_context *ctx = fz_new_context(NULL, NULL, FZ_STORE_UNLIMITED);
fz_register_document_handlers(ctx);
fz_document *doc = fz_open_document(ctx, "fichier.pdf");

int nb_pages = fz_count_pages(ctx, doc);

// Métadonnées (clés : "Title", "Author", "Subject", "Creator", "Producer")
char title[256];
fz_lookup_metadata(ctx, doc, "info:Title", title, sizeof(title));

// PDF chiffré
if (fz_needs_password(ctx, doc))
    fz_authenticate_password(ctx, doc, "mot_de_passe");
```

Charger une page

```
fz_page *page = fz_load_page(ctx, doc, 0); // index 0 = première page

fz_rect bounds = fz_bound_page(ctx, page);
```

```
float largeur = bounds.x1 - bounds.x0; // ex: 595.0 pour A4
float hauteur = bounds.y1 - bounds.y0; // ex: 842.0 pour A4

fz_drop_page(ctx, page);
```

Table des matières

```
fz_outline *toc = fz_load_outline(ctx, doc);
for (fz_outline *item = toc; item; item = item->next)
    printf("%s → page %d\n", item->title, item->page.page);
fz_drop_outline(ctx, toc);
```

Annotations

```
pdf_document *pdoc = pdf_document_from_fz_document(ctx, doc);
pdf_page *ppage = pdf_load_page(ctx, pdoc, 0);

for (pdf_annot *annot = pdf_first_annot(ctx, ppage);
     annot; annot = pdf_next_annot(ctx, annot))
{
    enum pdf_annot_type type = pdf_annot_type(ctx, annot);
    if (type == PDF_ANNOT_TEXT || type == PDF_ANNOT_FREE_TEXT)
        printf("Annotation : %s\n", pdf_annot_contents(ctx, annot));
}
pdf_drop_page(ctx, ppage);
```

4. Créer et modifier un PDF

Créer un PDF vierge

```
pdf_document *pdoc = pdf_create_document(ctx);

fz_rect mediabox = { 0, 0, 595, 842 }; // A4
pdf_obj *resources = pdf_new_dict(ctx, pdoc, 1);
fz_buffer *contents = fz_new_buffer(ctx, 1024);

// Contenu PDF (opérateurs de bas niveau)
fz_append_string(ctx, contents,
    "BT\n"
    "/F1 24 Tf\n"
    "100 700 Td\n"
    "(Bonjour MuPDF) Tj\n"
    "ET\n"
);
```

```
pdf_page *page = pdf_add_page(ctx, pdoc, mediabox, 0, resources, contents);
pdf_insert_page(ctx, pdoc, -1, pdf_page_obj(ctx, page));
```

Ajouter une police standard

```
// 14 polices standard disponibles : Helvetica, Times-Roman, Courier...
pdf_obj *font = pdf_add_simple_font(ctx, pdoc,
    fz_new_base14_font(ctx, "Helvetica"),
    PDF_SIMPLE_ENCODING_LATIN);
pdf_dict_puts(ctx, resources, "F1", font);
pdf_drop_obj(ctx, font);
```

Ajouter une image

```
fz_image *img = fz_new_image_from_file(ctx, "logo.png");
pdf_obj *img_obj = pdf_add_image(ctx, pdoc, img);

fz_append_string(ctx, contents,
    "q\n"
    "200 0 0 100 50 600 cm\n" // x=50, y=600, w=200, h=100
    "/Im1 Do\n"
    "Q\n"
);
fz_drop_image(ctx, img);
```

Modifier les métadonnées

```
pdf_document *pdoc = pdf_document_from_fz_document(ctx, doc);
pdf_obj *info = pdf_dict_get(ctx, pdf_trailer(ctx, pdoc), PDF_NAME(Info));
pdf_dict_put_string(ctx, info, PDF_NAME(Title), "Mon Document", -1);
pdf_dict_put_string(ctx, info, PDF_NAME(Author), "Mon Nom", -1);
```

Ajouter une annotation surlignage

```
pdf_page *ppage = pdf_load_page(ctx, pdoc, 0);
pdf_annot *annot = pdf_create_annot(ctx, ppage, PDF_ANNOT_HIGHLIGHT);

fz_rect rect = { 100, 700, 400, 720 };
pdf_set_annot_rect(ctx, annot, rect);
pdf_set_annot_color(ctx, annot, 3, (float[]){1.0f, 0.9f, 0.0f});
pdf_set_annot_contents(ctx, annot, "Passage important");
pdf_update_annot(ctx, annot);
pdf_drop_annot(ctx, annot);
```

Sauvegarder

```
pdf_write_options opts = { 0 };
opts.do_compress      = 1;
opts.do_compress_images = 1;
opts.do_garbage       = 2; // supprimer objets orphelins
// opts.do_linear      = 1; // linéariser pour le web

pdf_save_document(ctx, pdoc, "sortie.pdf", &opts);
```

5. Extraction de texte

Texte brut (le plus rapide)

```
fz_page *page = fz_load_page(ctx, doc, 0);
fz_stext_options opts = {};
opts.flags = FZ_STEXT_PRESERVE_WHITESPACE | FZ_STEXT_DEHYPHENATE;
// Autres flags : FZ_STEXT_PRESERVE_LIGATURES
//                FZ_STEXT_INHIBIT_SPACES
//                FZ_STEXT_PRESERVE_IMAGES

fz_stext_page *stext = fz_new_stext_page_from_page(ctx, page, &opts);

fz_buffer *buf = fz_new_buffer_from_stext_page(ctx, stext);
unsigned char *data;
size_t len = fz_buffer_storage(ctx, buf, &data);
std::string texte((char*)data, len);

fz_drop_buffer(ctx, buf);
fz_drop_stext_page(ctx, stext);
fz_drop_page(ctx, page);
```

Hierarchie structurée — bloc → ligne → caractère

```
for (fz_stext_block *block = stext->first_block; block; block = block->next) {
    if (block->type != FZ_STEXT_BLOCK_TEXT) continue;
    // block->bbox = rectangle englobant

    for (fz_stext_line *line = block->u.t.first_line; line; line = line->next)
    {
        // line->bbox, line->wmode (0=horizontal, 1=vertical), line->dir

        for (fz_stext_char *ch = line->first_char; ch; ch = ch->next) {
            printf("U+%04X  x=%.1f y=%.1f  size=%.1f\n",
                ch->c,           // codepoint Unicode
                ch->origin.x,    // position x
                ch->origin.y,    // position y
```

```

        ch->size);          // taille en points
    // ch->font  → fz_font* (nom, graisse, style)
    // ch->quad  → fz_quad (4 coins du glyphe)
    // ch->color → couleur RGBA packed
    }
}
}

```

Recherche plein texte

```

fz_quad hits[100];
int nb_hits = fz_search_page(ctx, page, "intelligence artificielle",
                             NULL, hits, 100);

for (int i = 0; i < nb_hits; i++) {
    fz_rect r = fz_rect_from_quad(hits[i]);
    printf("Trouvé à : (%.1f, %.1f) - (%.1f, %.1f)\n",
           r.x0, r.y0, r.x1, r.y1);
}

```

Export formaté

```

// Texte brut
fz_output *out = fz_new_output_with_path(ctx, "sortie.txt", 0);
fz_output_print_stext_page_as_text(ctx, out, stext);
fz_drop_output(ctx, out);

// HTML (conserve le layout)
fz_output *html = fz_new_output_with_path(ctx, "sortie.html", 0);
fz_print_stext_page_as_html(ctx, html, stext, 0);
fz_drop_output(ctx, html);

// JSON (utile pour pipeline IA)
fz_output *json = fz_new_output_with_path(ctx, "sortie.json", 0);
fz_print_stext_page_as_json(ctx, json, stext, 1.0f);
fz_drop_output(ctx, json);

```

6. Rendu en image

Rasteriser une page → PNG

```

fz_page *page = fz_load_page(ctx, doc, 0);
fz_rect bounds = fz_bound_page(ctx, page);

// 150 DPI (PDF natif = 72 DPI)

```

```

fz_matrix ctm = fz_scale(150.0f/72.0f, 150.0f/72.0f);
fz_irect bbox = fz_round_rect(fz_transform_rect(bounds, ctm));

fz_pixmap *pix = fz_new_pixmap_with_bbox(ctx,
    fz_device_rgb(ctx), bbox, NULL, 1);
fz_clear_pixmap_with_value(ctx, pix, 0xFF); // fond blanc

fz_device *dev = fz_new_draw_device(ctx, ctm, pix);
fz_run_page(ctx, page, dev, ctm, NULL);
fz_close_device(ctx, dev);
fz_drop_device(ctx, dev);

fz_save_pixmap_as_png(ctx, pix, "page_0.png");
// JPEG : fz_save_pixmap_as_jpeg(ctx, pix, "page_0.jpg", 90);

fz_drop_pixmap(ctx, pix);
fz_drop_page(ctx, page);

```

Accès aux pixels bruts (OpenCV / pipeline IA)

```

int largeur    = fz_pixmap_width(ctx, pix);
int hauteur    = fz_pixmap_height(ctx, pix);
int canaux     = fz_pixmap_components(ctx, pix); // 3=RGB, 4=RGBA
int stride     = fz_pixmap_stride(ctx, pix);
unsigned char *pixels = fz_pixmap_samples(ctx, pix);

// Passer à OpenCV sans copie
cv::Mat mat(hauteur, largeur,
    canaux == 4 ? CV_8UC4 : CV_8UC3,
    pixels, stride);

```

Display list — rendu mis en cache

```

// Parser la page une seule fois
fz_display_list *dl = fz_new_display_list(ctx, bounds);
fz_device *rec = fz_new_list_device(ctx, dl);
fz_run_page(ctx, page, rec, fz_identity, NULL);
fz_close_device(ctx, rec);
fz_drop_device(ctx, rec);

// Re-rasteriser plusieurs fois très rapidement
for (float dpi : {72.0f, 150.0f, 300.0f}) {
    fz_matrix m = fz_scale(dpi/72.0f, dpi/72.0f);
    fz_irect bbox = fz_round_rect(fz_transform_rect(bounds, m));
    fz_pixmap *pix = fz_new_pixmap_with_bbox(ctx, fz_device_rgb(ctx), bbox,
    NULL, 0);
    fz_device *dev = fz_new_draw_device(ctx, m, pix);
    fz_run_display_list(ctx, dl, dev, m, bounds, NULL); // ← pas fz_run_page
    fz_close_device(ctx, dev);
}

```



```

    fz_drop_device(ctx, dev);
    fz_drop_pixmap(ctx, pix);
}
fz_drop_display_list(ctx, dl);

```

7. Patterns performance

Un contexte par thread (obligatoire)

```

// Thread principal
fz_context *main_ctx = fz_new_context(NULL, NULL, FZ_STORE_UNLIMITED);
fz_register_document_handlers(main_ctx);

// Dans chaque thread worker
void worker(fz_context *main_ctx, int page_idx) {
    fz_context *ctx = fz_clone_context(main_ctx); // clone thread-safe
    fz_document *doc = fz_open_document(ctx, "fichier.pdf");
    // ... traitement ...
    fz_drop_document(ctx, doc);
    fz_drop_context(ctx); // libérer le clone
}

```

RAII wrapper — zéro fuite mémoire

```

template<typename T, void(*Drop)(fz_context*, T*)>
struct MuObj {
    fz_context *ctx;
    T *ptr = nullptr;

    MuObj(fz_context *c, T *p) : ctx(c), ptr(p) {}
    ~MuObj() { if (ptr) Drop(ctx, ptr); }
    T* operator->() { return ptr; }
    T* get() { return ptr; }
    MuObj(const MuObj&) = delete;
    MuObj& operator=(const MuObj&) = delete;
};

using MuDoc = MuObj<fz_document, fz_drop_document>;
using MuPage = MuObj<fz_page, fz_drop_page>;
using MuPix = MuObj<fz_pixmap, fz_drop_pixmap>;
using MuText = MuObj<fz_stext_page, fz_drop_stext_page>;

// Utilisation – libération automatique à la sortie du scope
MuDoc doc (ctx, fz_open_document(ctx, "fichier.pdf"));
MuPage page (ctx, fz_load_page(ctx, doc.get(), 0));

```

Contrôle du cache (embarqué)

```
// Limiter la mémoire cache
fz_context *ctx = fz_new_context(NULL, NULL, 16 * 1024 * 1024); // 16 MB

// Vider le cache manuellement
fz_empty_store(ctx);

// Connaître la taille actuelle
size_t taille = fz_store_size(ctx);
```

Pipeline IA — extraction sans copie

```
class PdfPipeline {
    fz_context *ctx;
    fz_document *doc;
public:
    PdfPipeline(const char *path) {
        ctx = fz_new_context(NULL, NULL, 32 * 1024 * 1024);
        fz_register_document_handlers(ctx);
        doc = fz_open_document(ctx, path);
    }
    ~PdfPipeline() {
        fz_drop_document(ctx, doc);
        fz_drop_context(ctx);
    }

    // Callback pour chaque bloc – zéro allocation intermédiaire
    void process(std::function<void(std::string_view, fz_rect)> cb) {
        fz_stext_options opts = { FZ_STEXT_DEHYPHENATE };
        for (int i = 0; i < fz_count_pages(ctx, doc); i++) {
            fz_page *page = fz_load_page(ctx, doc, i);
            fz_stext_page *stext =
                fz_new_stext_page_from_page(ctx, page, &opts);

            for (auto *b = stext->first_block; b; b = b->next) {
                if (b->type != FZ_STEXT_BLOCK_TEXT) continue;
                fz_buffer *buf = fz_new_buffer_from_stext_block(ctx, b);
                unsigned char *data; size_t len;
                len = fz_buffer_storage(ctx, buf, &data);
                cb(std::string_view((char*)data, len), b->bbox);
                fz_drop_buffer(ctx, buf);
            }

            fz_drop_stext_page(ctx, stext);
            fz_drop_page(ctx, page);
        }
    }
};

// Utilisation
PdfPipeline pipeline("doc.pdf");
```

```
pipeline.process([](std::string_view texte, fz_rect pos) {  
    tokenizer.encode(texte); // directement au modèle IA  
});
```

Récapitulatif des fonctions clés

Fonction	Rôle
<code>fz_new_context()</code>	Créer le contexte principal
<code>fz_clone_context()</code>	Cloner pour un thread
<code>fz_open_document()</code>	Ouvrir un fichier
<code>fz_count_pages()</code>	Nombre de pages
<code>fz_load_page()</code>	Charger une page
<code>fz_bound_page()</code>	Dimensions d'une page
<code>fz_new_stext_page_from_page()</code>	Extraire le texte structuré
<code>fz_search_page()</code>	Recherche plein texte
<code>fz_new_pixmap_with_bbox()</code>	Créer un pixmap
<code>fz_new_draw_device()</code>	Device de rendu
<code>fz_run_page()</code>	Rasteriser une page
<code>fz_new_display_list()</code>	Créer une liste d'affichage
<code>pdf_create_document()</code>	Nouveau PDF vierge
<code>pdf_add_page()</code>	Ajouter une page
<code>pdf_save_document()</code>	Sauvegarder
<code>pdf_create_annot()</code>	Créer une annotation